HexmodX
User Manual

BOSCH

Abstract

This document shall help the user of HexmodX to know how to work with the tool. The focus is set to enable the user to generate a modified hex file in a short time using **Command Line Switches** use case.

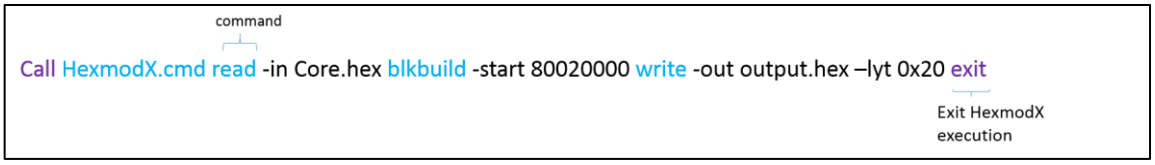
Table of Contents

HexmodX can be invoked using command line switches use case. Hex file can be read and various operations can be performed to modify the hex file. To know more about the feature you can use the command "***HexmodX.cmd -help***" and for information on a specific command, type "***HexmodX.cmd -help CommandName***" from the command line interface.

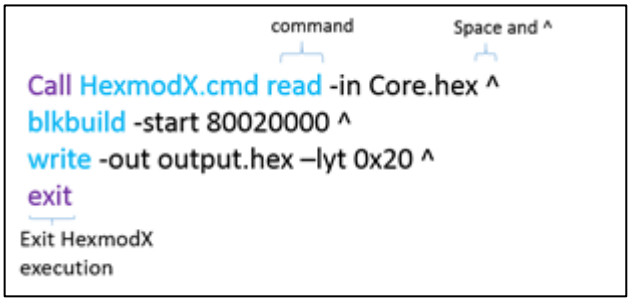
The commands can be invoked through **bat** file, **hmd** file or in Command prompt directly.

Execution from bat file

Single Line Command:



Multiple line command using " ^". Mainly to differentiate commands.



Execution from CLI

Commands can be executed from CLI through **single line command execution only**.

Following are brief explanation on the various commands supported.

1) Common Hex Operations

a) Basic Input - Output Operations

COMMAND NAME	DESCRIPTION
READ	Reads hex file of type ".hex", ".s19" and ".bin". READ command can also be used to read data from a hex file with a particular address and length.
SELECT	When multiple hex files are read, SELECT can be used to select one hex file to operate on.
WRITE	Writes output hex file of type ".hex", ".s19" and ".bin". WRITE command can also be used to write the data read using READ command at a particular address.
WRITERANGE	Serialize a range of data.

b) Patch Check View Operations

COMMAND NAME	DESCRIPTION
RESULT-LOC	Specifies location path to Patch/Check/View operation's resultant xml file.
PATCH	Patches data of type IHEX, ASCII or reference data at a particular address.
CHECK	Checks for data of type IHEX, ASCII or reference data at a particular address.
VIEW	View data at a particular address.

c) Memory Operations

COMMAND NAME	DESCRIPTION
MEMCOPY	Copies a range of data from source to destination address within same hex file.
MEMMOVE	Moves a range of data from source to destination address within same hex file.
MEMCOPYACROSS	Copies a range of data from source to destination address across files.
MEMMOVEACROSS	Moves a range of data from source to destination address across files.
MEMDELETE	Deletes a range of data.

d) Fill Operations

COMMAND NAME	DESCRIPTION
FILLV	Fills gaps within specified range with user defined byte or default value "0x00".
FILLP	Fills specified range with user defined byte value or default value "0x00".
FILLALL	Fills all the gaps in the hex file or from a given start address till the end of that file.

e) Infoblock Operations

COMMAND NAME	DESCRIPTION
BLKBUILD	Builds the already linked Infoblocks for processing.
BLKLINK	Links list of Infoblocks provided by the BLKLINK command.
DETAILEDLOG	Logs the basic infoblock structure into an xml file.
INFOTABLELOG	Logs the Infotable information of each Infoblock into an xml file.
DELETEBLOCK	Deletes an infoblock.
LOG	Logs information of the basis infoblock and checksum entries.
WRITECKS	Sets or tests Infoblock Checksum and ChecksumBlock Checksum.
SWID	Sets or test SWID field within each Infoblock.
SWTTNR	Sets or tests SWTTNR field within each Infoblock.
ENDPATTERN	Sets or tests ENDPATTERN field with "0xDEADBEEF".
COMPFC	Sets or tests compatibility between infoblocks and can be used to reset compatibility.
COMP	Lists the infoblocks whose compatibility must be set or tested.

f) ABM - BMI Operations

COMMAND NAME	DESCRIPTION
ABM	Performs ABM and BMI build.

g) Conversion Operations

COMMAND NAME	DESCRIPTION
CONVERT	Coverts supported hex file format into ".hex", ".s19" and ".bin" format.

h) Hex Compare Operations

COMMAND NAME	DESCRIPTION
HEXCOMPARE	Compares two hex files and generates cmp comparison report.

i) Checksum algorithms Operations

COMMAND NAME	DESCRIPTION
ADD8	ADD8 algorithm is selected to calculate the checksum.
ADD16	ADD16 algorithm is selected to calculate the checksum.
ADD32	ADD32 algorithm is selected to calculate the checksum.
CRC32	CRC32 algorithm is selected to calculate the checksum.
CRC64	CRC64 algorithm is selected to calculate the checksum.
CRC16CCITT	CRC16CCITT algorithm is selected to calculate the checksum.
CRC32CCITT	CRC32CCITT algorithm is selected to calculate the checksum.
CRCxx	CRCxx algorithm is selected to calculate the checksum.
IFXCRC	IFXCRC algorithm is selected to calculate the checksum.
MOD256	MOD256 algorithm is selected to calculate the checksum.
SHA256	SHA256 algorithm is selected to calculate the checksum.
SHA512	SHA512 algorithm is selected to calculate the checksum.

2) BaseIO Operations

a) READ

Reads hex file of type ".hex", ".s19" and ".bin".

read -in hexfile [-id ID] [-igCks {"true"|"false"}]

SWITCHES	DESCRIPTION
-in	"-in" takes input hex file to be read. Only Intel hex file is supported currently. Mandatory switch.
-id	Unique ID for the hex file read. If more than one hex file is read, this field is useful for HexmodX to recognize which hex file to operate on. Must begin with "ID". Optional switch.
-igcks	Ignores wrong line checksum of input hex file when set to TRUE. By default set to FALSE. Optional switch.
-type	Type of the hex file. Type can be IHEX, S19 and BIN. Optional switch. By default, file extension is considered.

Example:

```
read -in input.hex -id ID1 -igCks true -type IHEX ^
exit
```

Read command can also be used to read data from a hex file with a particular address and length. This read data can be useful when writing the data to another address also performing a copy operation.

read -addr address [-l len] [-fill fillGapValue]

SWITCHES	DESCRIPTION
-addr	Takes address value such as 0x80000000. Mandatory switch.
-l	Specifies length of data to be read. By default it reads only one byte. Optional switch.
- fill	Fills gaps with the one byte pattern provided. Optional switch.

Example:

```
read -in input.hex -igCks true ^
read -addr 0x80800000 -l 0x20 -fill 0xAE;
```

b) SELECT

The select operation is used when multiple hex files are read and one certain hex file must be operated upon. In such a case, the ID provided when reading multiple hex file is useful. If at all no ID is provided, the recently read hex file is considered by default.

select -id ID

SWITCHES	DESCRIPTION
-id	It chooses the corresponding hex file with specified ID to perform the operations on. Mandatory switch with Select command.

Example:

```
read -in input1.hex -id ID1 ^
read -in input2.hex -id ID2 ^      REM By default input2.hex file is operated on
select -id ID1 ^                   REM Selection is changed to input1.hex
exit
```

c) WRITE

Writes output hex file of type ".hex", ".s19" and ".bin".

write -out hexfilename [-lyt layout]

SWITCHES	DESCRIPTION
-out	Output hex file name. Mandatory switch.
-lyt	Specifies the layout that the output hex file must be serialized with. Default layout of 0x20. Optional switch.
-type	Type of the hex file. Type can be IHEX, S19 and BIN. Optional switch. By default, file extension is considered.

Example:

```
read -in input.hex ^
write -out outputFile.hex -lyt 0x10 -type IHEX ^
exit
```

Write command can also be used to write data to a hex file at a particular address. The data that was read using READ command can be written at the address by providing "readbytes" as the value, performing a copy operation. We can also specify the data you desire to write.

write -addr address [-val {data|"readbytes"}]

SWITCHES	DESCRIPTION
-addr	Takes address value such as 0x80000000. Mandatory switch.
-val	Specifies the value to be written. If read command was used to read a particular range of data, then "readbytes" are written at address 0x80800100. Ex.: -var readbytes OR -var 0x12345678

Example:

```
read -in input.hex ^
read -addr 0x80800000 -l 0x20 -fill 0xAE ^    REM "readBytes"
write -addr 0x80801000 -val readbytes ^       REM writes "readbytes".
write -addr 0x80802000 -val 0xAE123411 ^      REM writes 0xAE123411 at address.
write -out outputFile.hex -lyt 0x10 ^
exit
```

d) WRITERANGE

Serializes a range of data.

writeRange -addr address -l len

SWITCHES	DESCRIPTION
-addr	Start address. Mandatory switch.
-l	Length of data to be serialized. Mandatory switch.

Example:

```
read -in input.hex ^
writeRange -addr 0x8002003E -l 0x20 ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

e) MERGE HEX FILES

To merge two or more hex files.

Merge -count no. of files

-file file1_name -filetype file_type1
-file file2_name -filetype file_type2

-overwrite Boolean_value -igcks Boolean_value –mode Boolean_value.

Eg :

merge -count 2
-file output.bin -fileType bin
-file CB17.hex -fileType hex
-igcks true -overwrite true -mode littleendian.

SWITCHES	DESCRIPTION	Mandatory
-count	Count of total number of file to be merged. By default it is 1.	No
-file	Hex file name	Yes
-filetype	Hex file type	Yes
-overwrite	Set overwrite either with true or false. By default it is false.	No
-mode	Boolean attribute that specifies the Endianness. By default it is true. True -> little endian False -> big endian	No
-igcks	Boolean attribute to ignore line checksum. By default it is false	No

3) Patch Check View Operations

a) RESULT-LOC

Specifies location path to Patch/Check/View operation's resultant xml file.

result-loc -out logFilePath

SWITCHES	DESCRIPTION
-out	Output xml file path. Mandatory switch.

Example:

```
read -in medc18.hex -igCks true ^
result-loc -out patchCheckEnhance.xml ^
patch -t IHEX -addr 0x80030010 -val 0x123456 ^
check -t REF -addr 0x80030010 -val 0x800031000 -l 0x10 -n CheckData ^
view -t IHEX -addr 0x80030010 -val 0x800031000 -l 0x10 -n CheckData ^
write -out output.hex -lyt 0x10 ^
exit
```

b) PATCH

Patches data of type IHEX, ASCII or reference data at a particular address.

patch -t {"IHEX"|"ASCII"|"REF"} -addr address -val value [-l len] [-n name]

SWITCHES	DESCRIPTION
-t	Specifies type IHEX, REF or ASCII. Mandatory switch.
-addr	Address to be patched at. Mandatory switch.
-val	Specifies the value to be patched. If the type is REF, value at address specified in -val is considered for patching. Mandatory switch.
-l	For type IHEX and ASCII, length switch is an optional. If the type is REF then -l is a mandatory switch.

-n	Specifies the name of the command. Optional.

Example:

```
read -in medc18.hex -igCks true ^
result-loc -out patchCheckEnhance.xml ^
patch -t IHEX -addr 0x80030010 -val 0x123456 ^
patch -t REF -addr 0x80040020 -val 0x80020010 -l 0x20 ^
patch -t ASCII -addr 0x80030010 -val 12345678 -n "Patch Data" ^
write -out output.hex -lyt 0x10 ^
exit
```

c) CHECK

Checks for data of type IHEX, ASCII or reference data at a particular address.

check -t {"IHEX"|"ASCII"|"REF"} -addr address -val value [-l len] [-n name]

SWITCHES	DESCRIPTION
-t	Specifies type IHEX, REF or ASCII. Mandatory switch.
-addr	Address at which the data is checked. Mandatory switch.
-val	Specifies the value to be checked. If the type is REF, value at address specified in -val is considered for comparison. Mandatory switch.
-l	For type IHEX and ASCII, length switch is an optional. If the type is REF then -l is a mandatory switch.
-n	Specifies the name of the command. Optional.

Example:

```
read -in medc18.hex -igCks true ^
result-loc -out patchCheckEnhance.xml ^
check -t IHEX -addr 0x80030010 -val 0x123456 ^
check -t REF -addr 0x80040020 -val 0x80020010 -l 0x20 ^
check -t ASCII -addr 0x80030010 -val 12345678 -n "Check Data" ^
write -out output.hex -lyt 0x10 ^
exit
```

d) VIEW

View data at a particular address.

view -t {"IHEX"|"ASCII"} -addr address -l len [-n name]

SWITCHES	DESCRIPTION
-t	Specifies type IHEX, REF or ASCII. Mandatory switch.
-addr	Address at which data of specified length is viewed. Mandatory switch.
-l	Specifies the length to be viewed. Mandatory switch.
-n	Specifies the name of the command. Optional.

Example:

```
read -in medc18.hex -igCks true ^
result-loc -out patchCheckEnhance.xml ^
view -t IHEX -addr 0x80030010 -l 0x60 ^
check -t ASCII -addr 0x80030010 -l 0x32 -n "View Data" ^
write -out output.hex -lyt 0x10 ^
exit
```

4) Memory Operations

a) MEMCOPY

Copies a range of data from source to destination address within same hex file.

memCopy -from fromAddress -l len -to toAddress -o {"true"|"false"} [-fill fillValue]

SWITCHES	DESCRIPTION
-from	Specifies address from where data has to be copied from. Mandatory switch.
-l	Length of data to be copied. Mandatory switch.
-to	Specifies address at which copied data is written to. Mandatory switch.
-o	Set overwrite either with true or false. Mandatory switch.
-fill	Fills gaps with the one byte pattern provided. Optional switch.

Example:

```
read -in input1.hex ^
memCopy -from 0x80801000 -l 0x10 -to 0x80802000 -o true -fill 0xAA ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

b) MEMMOVE

Moves a range of data from source to destination address within same hex file.

memMove -from fromAddress -l len -to toAddress -o {"true"|"false"} [-fill fillValue]

SWITCHES	DESCRIPTION
-from	Specifies address from where data has to be moved from. Mandatory switch.
-l	Length of data to be moved. Mandatory switch.
-to	Specifies address at which data is written to. Mandatory switch.
-o	Set overwrite either with true or false. Mandatory switch.
-fill	Fills gaps with the one byte pattern provided. Optional switch.

Example:

```
read -in input1.hex ^
memMove -from 0x80801000 -l 0x10 -to 0x80802000 -o true -fill 0xAA ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

c) MEMDELETE

Deletes a range of data.

memDelete -from address -l len

SWITCHES	DESCRIPTION
-from	Address from which data should be deleted. Mandatory switch.
-l	Specifies length of data to be deleted. Mandatory switch.

Example:

```
read -in input1.hex ^
memDelete -addr 0x80801000 -l 0x10 ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

d) MEMCOPYACROSS

Copies a range of data from source to destination address across files.

memCopyAcross -id ID1 -from fromAddress -l len -id2 ID2 -to toAddress -o {"true"|"false"} [-fill fillValue]

SWITCHES	DESCRIPTION
-id	ID value of the source hex file. ID value can be associated with a hex file using READ command.
-from	Specifies source hex file address from where data has to be copied from. Mandatory switch.
-l	Length of data to be copied. Mandatory switch.
-id2	ID value of the destination hex file. ID value can be associated with a hex file using READ command.
-to	Specifies destination hex file address at which copied data is written to. Mandatory switch.
-o	Set overwrite either with true or false. Mandatory switch.
-fill	Fills gaps with the one byte pattern provided. Optional switch.

Example:

```
read -in input1.hex ^
memCopyAcross -id ID1 -from 0x80801000 -l 0x10 -id2 ID2 -to 0x80802000 -o true -fill 0xAA ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

e) MEMMOVEACROSS

Moves a range of data from source to destination address across files.

memMoveAcross -id ID1 -from fromAddress -l len -id2 ID2 -to toAddress -o {"true"|"false"} [-fill fillValue]

SWITCHES	DESCRIPTION
-id	ID value of the source hex file. ID value can be associated with a hex file using READ command.
-from	Specifies source hex file address from where data has to be moved from. Mandatory switch.

-l	Length of data to be moved. Mandatory switch.
-id2	ID value of the destination hex file. ID value can be associated with a hex file using READ command.
-to	Specifies destination hex file address at which data is written to. Mandatory switch.
-o	Set overwrite either with true or false. Mandatory switch.
-fill	Fills gaps with the one byte pattern provided. Optional switch.

Example:

```
read -in input1.hex -igcks true ^
memMoveAcross -id ID1 -from 0x80801000 -l 0x10 -id2 ID2 -to 0x80802000 -o true -fill 0xAA ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

5) Fill Operations

a) FILLV

Fills gaps within specified range with user defined byte or default value "0x00".

fillv -addr address -l len [-fill fillValue]

SWITCHES	DESCRIPTION
-addr	Start address from where the gaps must be filled for specified length. Mandatory switch.
-l	Specifies length for fill operation. Mandatory switch.
-fill	One byte value. Default value 0x00. Optional field.

Example:

```
read -in input1.hex ^
fillv -addr 0x80801000 -l 0x10 -fill 0xAA ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

b) FILLP

Fills specified range with user defined byte value or default value "0x00".

fillp -addr address -l len [-fill fillValue]

SWITCHES	DESCRIPTION
-addr	Start address from where a particular pattern must be filled for specified length. Mandatory switch.
-l	Specifies length for fill operation. Mandatory switch.
-fill	One byte value. Default value 0x00. Optional field.

Example:

```
read -in input1.hex ^
fillp -addr 0x80001000 -l 0x20 -fill 0xFF ^
write -out outputFile.hex -lyt 0x10 ^
exit
```

c) FILLALL

Fills all the gaps in the hex file or from a given start address till the end of that file.

fillAll -t {"all"|address} [-fill fillValue]

SWITCHES	DESCRIPTION
-t	Takes "all" or address as value. "all" means gaps in the complete hex file will be filled. If address is provided, gaps from that address till the end will be filled with fill value. Mandatory switch.
-fill	One byte value. Default value 0x00. Optional field.

Example:

1)
```
read -in input1.hex ^
fillAll -t all -fill 0xAF ^          REM Fills all the gaps in the hex file.
write -out outputFile.hex -lyt 0x10 ^
exit
```

2)
```
read -in inutp2.hex ^
fillAll -t 0x80020000 -fill 0xFF ^   REM Fills gaps from the specified address until the end.
write -out outputFile.hex -lyt 0x10 ^
exit
```

6) Infoblock Operations

a) BLKBUILD

Builds the already linked Infoblocks for processing.

blkBuild -start startAddress

SWITCHES	DESCRIPTION
-start	Start address of the first infoblock in the link chain. Mandatory switch.

Example:

```
read -in medc18.hex -id ID1 -igCks true ^
blkBuild -start 0x80020000 ^
write -out output.hex ^
exit
```

b) BLKLINK

Links list of Infoblocks provided by the BLKLINK command.

blkLink -start startAddress -next nextBlkAddress

SWITCHES	DESCRIPTION
-start	Start address of the infoblock. Mandatory switch.
-next	Start address of the next infoblock to be in the link chain.

Example:

```
read -in medc18.hex -id ID1 -igCks true ^
blkLink -start 0x80020000 -next 0x80100000 ^
blkLink -start 0x80100000 -next 0x80200000 ^
blkLink -start 0x80200000 -next 0x0 ^
writecks -fc test ^                    REM Tests the checksum values.
write -out output.hex ^
exit
```

c) DETAILEDLOG

detailedlog -out filepath -log {address|"all"|blockName}

SWITCHES	DESCRIPTION
-out	Output address model xml file path.
-log	Logs detailed infoblock information for specified block or all the blocks. It takes values such as start address of a block, "all" to log all the infoblock information, or blockname such as "ASW0". Mandatory switch.
blockName	HexmodX supports different memory blocks such as, ASW0, ASW1, ASW10, ASW11, ASW12, ASW13, ASW14, ASW15, ASW2, ASW3, ASW4, ASW5, ASW6, ASW7, ASW8, ASW9, CB, CB1, CB10, CB11, CB12, CB13, CB14, CB15, CB2, CB3, CB4, CB5, CB6, CB7, CB8, CB9, CTP, CUST, DS0, DS1, DS10, DS11, DS12, DS13, DS14, DS15, DS2, DS3, DS4, DS5, DS6, DS7, DS8, DS9, DSC0, EcuSecuSrv0, EcuSecuSrv1, EcuSecuSrv10, EcuSecuSrv11, EcuSecuSrv12, EcuSecuSrv13, EcuSecuSrv14, EcuSecuSrv15, EcuSecuSrv2, EcuSecuSrv3, EcuSecuSrv4, EcuSecuSrv5, EcuSecuSrv6, EcuSecuSrv7, EcuSecuSrv8, EcuSecuSrv9, HSM0, HSM1, HSM10, HSM11, HSM12, HSM13, HSM14, HSM15, HSM2, HSM3, HSM4, HSM5, HSM6, HSM7, HSM8, HSM9, PBC0, SB, SECUMGR, SWC0, UNKNOWN, VDS, VDS1, VDS10, VDS11, VDS12, VDS13, VDS14, VDS15, VDS2, VDS3, VDS4, VDS5, VDS6, VDS7, VDS8, VDS9.

Example:

```
read -in medc18.hex ^
blkbuild -start 0x80020000 ^
detailedlog -out AddressModel.xml -log 0x80100000 ^
detailedlog -out AddressModel.xml -log all ^
detailedlog -out AddressModel.xml -log ASW0 ^
write -out output.hex -lyt 0x20 ^
exit
```

d) INFOTABLELOG

Logs the Infotable information of each Infoblock into an xml file.

infotablelog -out filepath -log {address|"all"|blockName}

SWITCHES	DESCRIPTION
-out	Output infotable xml file path.
-log	Logs infotable information of a specified block or all the blocks. It takes values such as start address of a block, "all" to log all the infoblock information, or blockname such as "ASW0". Mandatory switch.
blockName	HexmodX supports different memory blocks such as, ASW0, ASW1, ASW10, ASW11, ASW12, ASW13, ASW14, ASW15, ASW2, ASW3, ASW4, ASW5, ASW6, ASW7, ASW8, ASW9, CB, CB1, CB10, CB11, CB12, CB13, CB14, CB15, CB2, CB3, CB4, CB5, CB6, CB7, CB8, CB9, CTP, CUST, DS0, DS1, DS10, DS11, DS12, DS13, DS14, DS15, DS2, DS3, DS4, DS5, DS6, DS7, DS8, DS9, DSC0, EcuSecuSrv0, EcuSecuSrv1, EcuSecuSrv10, EcuSecuSrv11, EcuSecuSrv12, EcuSecuSrv13, EcuSecuSrv14, EcuSecuSrv15, EcuSecuSrv2, EcuSecuSrv3, EcuSecuSrv4, EcuSecuSrv5, EcuSecuSrv6, EcuSecuSrv7, EcuSecuSrv8, EcuSecuSrv9, HSM0, HSM1, HSM10, HSM11, HSM12, HSM13, HSM14, HSM15, HSM2, HSM3, HSM4, HSM5, HSM6, HSM7, HSM8, HSM9, PBC0, SB, SECUMGR, SWC0, UNKNOWN, VDS, VDS1, VDS10, VDS11, VDS12, VDS13, VDS14, VDS15, VDS2, VDS3, VDS4, VDS5, VDS6, VDS7, VDS8, VDS9.

Example:

```
read -in medc18.hex ^
blkbuild -start 0x80020000 ^
infotablelog -out AddressModel.xml -log 0x80100000 ^
infotablelog -out AddressModel.xml -log all ^
infotablelog -out AddressModel.xml -log ASW0 ^
write -out output.hex -lyt 0x20 ^
exit
```

e) DELETEBLOCK

Deletes an infoblock.

deleteBlock -igErr {"true"|"false"} -blkid blockId -igRelink {"true"|"false"}

SWITCHES	DESCRIPTION
-igerr	If value is "true" then even if the block is not present in the hex file, it is ignored. If it is "false", HexmodX aborts with an error saying the block is not present in the file. Mandatory switch.
-blkid	Block Id of the i18nfoblock to be deleted. Mandatory switch.
-igrelink	If value is "true" then after the deletion of the info block, all other blocks will NOT be relinked. If it is "false", HexmodX will relink all the remaining blocks in the same order as it was linked earlier. Mandtory switch.

Example:

```
read -in medc18.hex ^
deleteblock -blkID 0x61 -igErr false -igRelink false ^
write -out output.hex -lyt 0x20 ^
exit
```

f) LOG

Logs information of the basis infoblock and checksum entries.

log [-ib filename] [-ck filename]

SWITCHES	DESCRIPTION
-ib	Output file path. Logs basis infoblock information. Optional switch.
-ck	Output file path. Logs checksum entry information. Optional switch.

Example:

```
read -in medc18.hex ^
blkbuild -start 0x80020000 ^
log -ib ibBefore.txt -ck cksBefore.xml ^
exit
```

g) WRITECKS

Sets or tests Infoblock Checksum and ChecksumBlock Checksum.

writecks -fc {"test"|"set"}

SWITCHES	DESCRIPTION
-fc	Takes values "set" or "test". "set" to update the checksum values and "test" to test if checksum values are correct. Mandatory switch.

Example:

```
read -in medc18.hex -id ID1 -igCks true ^
blkLink -start 0x80020000 -next 0x80100000 ^
blkLink -start 0x80100000 -next 0x80200000 ^
blkLink -start 0x80200000 -next 0x0 ^
writecks -fc test ^
write -out output.hex ^
exit
```

h) SWID

Sets or test SWID field within each Infoblock.

swid -val value -fc {"test"|"set"} -t {"IHEX"|"ASCII"} [-addr blockaddress]

SWITCHES	DESCRIPTION
-val	Values to be patched or tested.
-fc	Takes values "set" or "test". "set" to update the swid value and "test" to test correct swid value. Mandatory switch.
-t	Defines what type of data to be patched or tested. Values can be "IHEX" or "ASCII".
-addr	Provides the start address of Infoblock for which specified operation should be performed. Optional switch.

Example:

```
read -in medc18.hex -igCks true ^
blkBuild -start 0x80020000 ^
swid -val 0x1234567800010203 -fc set -addr 0x08FC0020 -t hex ^
write -out output.hex -lyt 0x20 ^
exit
```

i) SWTTNR

Sets or tests SWTTNR field within each Infoblock.

swttnr -val value -fc {"test"|"set"} -t {"IHEX"|"ASCII"} [-addr blockaddress]

SWITCHES	DESCRIPTION
-val	Values to be patched or tested.
-fc	Takes values "set" or "test". "set" to update the swttnr value and "test" to test correct swttnr value. Mandatory switch.
-t	Defines what type of data to be patched or tested. Values can be "IHEX" or "ASCII".
-addr	Provides the start address of Infoblock for which specified operation should be performed. Optional switch.

Example:

```
read -in medc18.hex -igCks true ^
blkBuild -start 0x80020000 ^
swid -val 0x1234567800010203 -fc set -addr 0x08FC0020 -t hex ^
swttnr -val 0xAFAFAFAFAFAFAFAFAFAF -fc set -t hex -addr 0x80020000 ^
write -out output.hex -lyt 0x20 ^
exit
```

j) VALIDPATTERN

Sets or tests VALIDPATTERN field with 0xDEADBEEF

validpattern -fc {"test"|"set"}

Note: command **endpattern** is deprecated.

SWITCHES	DESCRIPTION
-fc	Takes values "set" or "test". "set" to update the endpattern value with 0xDEADBEEF and "test" to test the field value with 0xDEADBEEF pattern. Mandatory switch.

Example:

```
read -in medc18.hex ^
blklink -start 0x8FC0020 -next 0x8FD0000 ^
blklink -start 0x8FD0000 -next 0x0 ^
validpattern -fc set ^
write -out output.hex -lyt 0x20 ^
exit
```

k) COMPFC

Sets or tests compatibility between infoblocks and can be used to reset compatibility.

***This command must be used before COMP command.

compFc -fc {"test"|"set"} [-rst {"true"|"false"}]

SWITCHES	DESCRIPTION
-fc	"set" triggers the update of compatibility based on dependency link provided by "comp" command. "test" tests the already set compatibility. Mandatory switch.
-rst	Reset compatibility between all the infoblocks. Optional switch.

Example:

```
read -in medc18.hex ^
blklink -start 0x0060C100 -next 0x00610100 ^
blklink -start 0x00610100 -next 0x0 ^
writecks -fc set ^
compFc -fc set ^
comp -addr 0x00610100 -depaddr 0x0060C100 ^
write -out output.hex -lyt 0x20 ^
exit
```

l) COMP

comp -addr blkAddress -depaddr depBlkAddress

SWITCHES	DESCRIPTION
-addr	Start address of the infoblock that is compatible with next block.
-depaddr	Start address of the next compatibility dependent infoblock.

Example:

```
read -in medc18.hex ^
blklink -start 0x0060C100 -next 0x00610100 ^
blklink -start 0x00610100 -next 0x0 ^
writecks -fc set ^
compFc -fc set ^
comp -addr 0x00610100 -depaddr 0x0060C100 ^
write -out output.hex -lyt 0x20 ^
exit
```

7) ABM and BMI build Operations

Performs ABM and BMI build.

abm -primary address [-bmiprimary address] [-bmisec address] [-device {DEV5,TC3XX}]

SWITCHES	DESCRIPTION
-primary	Start address of ABM primary location. Mandatory switch.
-sec	Deprecated
-exprimary	Deprecated

-exsec	Deprecated
-bmiprimary	Start address of BMI primary location. Optional Switch.
-bmisec	Start address of BMI secondary location. Optional Switch.
-device	Takes value "DEV5" and "TC3XX", specifying the project type. Optional field.

Example:

1)
```
read -in rba_BootCtrl_hexmod.hex -igcks true ^
abm -primary 0x80000999 -sec 0x80000000 -exprimary 0x8080FFE0 -exsec 0x8088FFE0 ^
write -out _out/output_abm.hex -lyt 0x10 ^
exit
```

2)
```
read -in rba_BootCtrl_hexmod.hex -igcks true ^
abm -primary 0x800F0000 -bmiprimary 0xAF400000 -bmisec 0xAF400200 ^
write -out _out/output_abm.hex -lyt 0x20 ^
exit
```

3)
```
read -in Test_TC39X.hex -igcks true ^
abm -primary 0x80000000 -device DEV5 ^
write -out _out/output_abm.hex -lyt 0x10 ^
exit
```

8) Convert Command

Coverts supported hex file format into ".hex", ".s19" and ".bin" format.

convert -in inputFilePath [-igcks {true|false}] -out outputFilePath [-lyt layout]

SWITCHES	DESCRIPTION
-in	Input hex file with extension ".hex", ".s19" or ".bin".
-igcks	Ignores wrong line checksum of input hex file when set to TRUE. By default set to FALSE. Optional switch.
-out	Output hex file path for conversion with appropriate extension of type INTEL(".hex"), MOTOROLA(".s19") and BIN(".bin"). Mandatory switch.
-lyt	Specifies the layout that the output hex file must be serialized with. Default layout of 0x20. Optional switch.

Example:

```
REM Converts intel(.hex) file to Motorola(.s19) hex file format.
convert -in input.hex -igcks true -out output.s19 -lyt 0x10 ^
exit
```

9) HexCompare Command

Compares two hex files and generates an xml comparison report.

hexcompare -file1 filePath -file2 filepath -log filepath [-exit {true|false}]

SWITCHES	DESCRIPTION
-file1	First input hex file for comparison.
-file2	Second input hex file for comparison.

-log	CMP Comparison report path.
-exit	Exits HexmodX if EXIT is "true" and if there are any difference found. It is "false" by default.

Example:

```
hexcompare -file1 input1.hex -file2 input2.hex -log HexCompareReport.cmp
exit
```

10) Checksum Algorithms command

Calculates checksum based on various checksum algorithms for the input hex files for given ranges. Various checksum algorithms supported in HexmodX are ADD8, ADD16, ADD32, CRC32, CRC64, CRC16CCITT, CRC32CCITT, IFXCRC, CRCxx, MOD256, SHA256, SHA512.

Syntax:

```
AlgorithmName -startaddress {start address} -endaddress {end address}
    -startvalue {start value} - endvalue {end value}
    -inverse {inverse flag} -alignment {alignment flag}
    –outputaddress {output address} –outputtxt {output text file name with path}
```

SWITCHES	DESCRIPTION	MANDATORY
-startaddress	Start address of the data range.	YES
-endaddress	End address of the data range.	YES
-startvalue	Start value for calculating checksum. Default start value is 0xFFFFFFFF.	NO
-endvalue	Start value for calculating checksum. Default start value is 0xFFFFFFFF.	NO
-inverse	Inverse flag which is a Boolean attribute. Default value is false.	NO
-alignment	Alignment flag which is a Boolean attribute. Default value is false.	NO
-outputaddress	Output address where the calculated checksum is patched.	NO
-outputtxt	Text file to be generated which contains the calculated checksum for given range.	NO

Example:

```
read -in Core.hex -igCks true
CRC32  -inverse  true –alignment true -startAddress 0x80820000 -endAddress 0x8082001F
       -startValue 0xFADACAFE  -endValue 0xFADACAFE
       -outputtxt chksum_hexmodX.txt   -outputaddress  0x80820040
```

Checksum value can also be calculated for **multiple ranges**.

Example:

```
read -in Core.hex -igCks true
ADD8 -startAddress 0x80820000 -endAddress 0x8082001F
```

-startAddress 0x80820020 -endAddress 0x8082003F
-outputtxt chksum_hexmodX.txt -outputaddress 0x80820040

Note that checksum value of first data range is considered as start value of second data range. Similarly for successive data ranges.

11) Compression and decompression command:

COMPRESS COMMAND

Performs Compression for given block of data

Below is the syntax :

compress -startaddress 0x8000 -endaddress 0x8160 -at 0x00010000 -outputfile output.bin

SWITCH	DESCRIPTION	MANDATORY
-startaddress	Start address to be compressed.	No
-endaddress	End address to be compressed.	No
-at	Destination address where the compressed data will be placed.	No
-outputfile	Output file name of the compressed hex file.	Yes
-type	Type of input HEX file.	No
-fillpattern	Pattern to be filled in the gaps.	No
-range	Specifies the number of data ranges to be compressed.	No
-lyt	Layout of the output hex file.	No

Example:

1) read -in test.hex -igCks true compress -at 0x00010000 -outputfile output.bin

2) read -in test.hex -igCks true compress -startaddress 0x8000 -endaddress 0x8160 -at 0x00010000 -type HEX -outputfile output.bin -lyt 0x10

3) read -in test.hex -igCks true compress -range 2 -startaddress 0x8000 -endaddress 0x8160 -startaddress 0x8160 -endaddress 0x82f60 -at 0x00010000 -type HEX -outputfile output.bin -lyt 0x10

DECOMPRESS COMMAND

Performs Decompression for given block of data

Below is the syntax :

decompress -startaddress 0x8000 -endaddress 0x8160 -at 0x00010000 -outputfile output.bin

SWITCH	DESCRIPTION	MANDATORY
-startaddress	Start address to be decompressed.	No
-endaddress	End address to be decompressed.	No
-at	Destination address where the decompressed data will be placed.	No

-outputfile	Output file name of the decompressed hex file.	Yes
-type	Type of input HEX file.	No
-fillpattern	Pattern to be filled in the gaps.	No
-range	Specifies the number of data ranges to be decompressed.	No
-lyt	Layout of the output hex file.	No

Example:

1) read -in test.hex -igCks true decompress -at 0x00010000 -outputfile output.bin

2) read -in test.hex -igCks true decompress -startaddress 0x8000 -endaddress 0x8160 -at 0x00010000 -type HEX -outputfile output.bin -lyt 0x10

3) read -in test.hex -igCks true decompress -range 2 -startaddress 0x8000 -endaddress 0x8160 -startaddress 0x8160 -endaddress 0x82f60 -at 0x00010000 -type HEX -outputfile output.bin -lyt 0x10

12) CLEANUP COMMAND:

Performs Cleanup for given block address.

Below is the syntax :

cleanup -blockaddress [blockaddress] -filename [filename] -type [typeof hex file]

SWITCHES	DESCRIPTION	MANDATORY SWITCH
-blockaddress	Start address of the block in input hex file	Yes
- filename	Output file name.	
-type	Type of output HEX file. By default it is IHEX	

Example:

1) read -in medc18_tmp.hex -type IHEX cleanup -blockaddress 0x80000020 -filename output.hex -type IHEX

2) read -in medc18_tmp.hex -type IHEX cleanup -blockaddress 0x80000020 -filename output.hex

13) ENCRYPT AND DECRYPT COMMAND:

ENCRYPTION COMMAND:

Performs Encryption for given block of data

Below is the syntax :

encrypt -startaddress 0x8000 -endaddress 0x8160 -algo [algorithm name] -key [key value] -vector [vector value]

SWITCHES	DESCRIPTION	MANDATORY
-startaddress	Start address to be encrypted.	Yes
-endaddress	End address to be encrypted.	Yes
-key	Key value.	No

-vector	Vector value.	No
-algo	Type of algorithm.	Yes

Example:

1) read -in input.hex -igCks true encrypt -startaddress 0x80030000 -endaddress 0x80030500 -algo AES/CBC/PKCS5Padding -key 0xffffffffffffffffffffffffffffffffF write -out output.hex -lyt 0x10

2) read -in input.hex -igCks true encrypt -startaddress 0x80030000 -endaddress 0x80030500 -algo AES/CBC/PKCS5Padding -key 0xffffffffffffffffffffffffffffffffF encrypt -startaddress 0xD8000000 -endaddress 0xD8000400 -algo AES/CBC/PKCS5Padding -key 0xffffffffffffffffffffffffffffffffF write -out output.hex -lyt 0x10

3) read -in input.hex -igCks true encrypt -startaddress 0x80030000 -endaddress 0x80030500 -algo AES/CBC/PKCS5Padding write -out output.hex -lyt 0x10

DECRYPTION COMMAND:

Performs Decryption for given block of data

Below is the syntax :

decrypt -startaddress [startaddress] -endaddress [endaddress] -algo [algorithm name] -key [key value] -vector [vector value]

SWITCHES	DESCRIPTION	MANDATORY
-startaddress	Start address to be decrypted.	Yes
-endaddress	End address to be decrypted.	Yes
-key	Key value.	No
-vector	Vector value.	No
-algo	Type of algorithm.	Yes

Example:

1) read -in input.hex -igCks true decrypt -startaddress 0x80030000 -endaddress 0x80030500 -algo AES/CBC/NoPadding -key 0xffffffffffffffffffffffffffffffffF write -out output.hex -lyt 0x10

2) read -in input.hex -igCks true decrypt -startaddress 0x80030000 -endaddress 0x80030500 -algo AES/CBC/NoPadding -key 0xffffffffffffffffffffffffffffffffF decrypt -startaddress 0xD8000000 -endaddress 0xD8000400 -algo AES/CBC/NoPadding -key 0xffffffffffffffffffffffffffffffffF write -out output.hex -lyt 0x10

3) read -in input.hex -igCks true decrypt -startaddress 0x80030000 -endaddress 0x80030500 -algo AES/CBC/NoPadding write -out output.hex -lyt 0x10

14) VARIANT CODDING COMMAND:

VDSIN COMMAND:

Performs Variant coding back for given infoblock.

Below is the syntax :

vdsback -type [file_type] -count [total number of input file] -name [file_name] -infoblockaddress [infoblockaddress] -vctrinfoblockaddress [vector block address] -vdslog [vds log file name] -vectortablelog [vector table log] -vectorinfolog [vectorinfo log] -fillvalue [fill value]

SWITCHES	DESCRIPTION	MANDATORY
-type	Type of the hex file	No
-name	Input file name.	Yes
-infoblockaddress	start address of the infoblock.	Yes
-fillvalue	Values to be filled in the gaps.	No
-vctrinfoblockaddress	Vector infoblock address.	No
-vdslog	Logging for VDS.	No
-vectortablelog	Logging for Vector table.	No
-vectorinfolog	Logging for Vector infotable.	No
-count	Total number of input file count.	No

Example:

1) vdsin -igcks true -count 5 -type IHEX -name mx18_make.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 -name mx18_make_1.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 -name mx18_make_2.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 -name mx18_make_3.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 -name mx18_make_4.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 -vdslog vdslog.log -vectortablelog vectortable.log -vectorinfolog vectorinfo.log write -out output.hex -lyt 0x10

2) vdsin -igcks true -count 2 -type IHEX -name mx18_make.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 -name mx18_make_1.hex -infoblockaddress 0x80300000 -vctrinfoblockaddress 0x80300000 write -out output.hex -lyt 0x10

VDSBACK COMMAND:

Performs Variant coding back for given infoblock.

Below is the syntax :

vdsback -startaddress 0x8000 -endaddress 0x8160 -at 0x00010000 -outputfile output.bin

SWITCHES	DESCRIPTION	MANDATORY
-type	Type of the hex file	No
-name	Input file name.	Yes
-infoblockaddress	start address of the infoblock.	Yes
-fillvalue	Values to be filled in the gaps.	No
-vctrinfoblockaddress	Vector infoblock address.	No
-vdslog	Logging for VDS.	No

Example:

1) read -in input.hex vdsback -type IHEX -name medc18_output_hexmodX.hex -infoblockaddress 0x09200000 -vdslog vds.log -fillvalue 0x00

2) read -in input.hex vdsback -type IHEX -name medc18_output_hexmodX.hex -infoblockaddress 0x09200000 -vctrinfoblockaddress 0x09200000 -vdslog vds.log -fillvalue 0x00

15) MD1TPROT MODULE:

a) HASH

Generate hash file for the given hex file using MDG1TPROT

Below is the syntax :

HASH -input [input hex file name] -output [output hash text file name] -igcks [true or false] -id [tag value] -filetype [input file type] -mode [bigendian or littleendian] -oidpub [OID value] -rangecount [range count] -startaddress [start address] -endaddress [endaddress] -infoblockcount [infoblock count] -infoblockaddress [infoblock start address] -tctype [trust center type] -HSMfile [hsm file name] -HSMconfig [HSM configuration] -division[division type]

SWITCHES	DESCRIPTION	MANDATORY
-input	Input hex file name.	Yes
-output	Output Hash file name.	Yes
-igcks	Ignores line checksum while parsing the input hex file.	No
-filetype	Input hex file type.	No
-id	ID given to the hash for of each range.	No
-memoryType	Memory type.	No
-division	Specifies the division type.	No
-HSMfile	HSM file name.	No
-HSMconfig	HSM configuration type.	No
-tctype	Trust center type Eg. BOSCH TRUST CENTER, VW TRUST CENTER	No
-mode	Mode type eg. Online or Offline.	No
-oidpub	OID value.	No
-infoblockaddress	Infoblock start address.	No
-infoblockcount	Infoblock count for which hash value to be calculated.	No

Example:

1) HASH -input medc18.hex -output output_hash.txt -igcks true -id tag1 -filetype ihex -mode bigendian -oidpub 1.3.6.1.4.1.3210.1178.20.0.1.1.1 -infoblockcount 2 -infoblockaddress 0x80020000 -infoblockaddress 0x80020000 -tctype VW_TRUST_CENTER

2) HASH -input medc18.hex -output output_hash.txt -igcks true -id tag1 -filetype ihex -mode bigendian -oidpub 1.3.6.1.4.1.3210.1178.20.0.1.1.1 -rangecount 2 -startaddress 0x80020000 -endaddress 0x80020100 -startaddress 0x80030000 -endaddress 0x80030100

b) INPUTHASH

Generate sign text file for the given hash text file using MDG1TPROT module.

Below is the syntax :

inputhash -input [input hex file name] -output [output hash text file name] -igcks [true or false] -id [tag value] -filetype [input file type] -mode [bigendian or littleendian] -oidpub [OID value] -rangecount [range count] -startaddress [start address] -endaddress [endaddress] -infoblockcount [infoblock count] -infoblockaddress [infoblock start address] -tctype [trust center type] -HSMfile [hsm file name] -HSMconfig [HSM configuration] -division[division type]

Example:

1) inputhash -inputtxt hash.txt -mode online -outputtext Sign.txt

2) inputhash -inputtxt hash.txt -mode online -outputtext Sign1.txt inputhash -inputtxt hash1.txt -mode online -outputtext Sign2.txt

SWITCHES	DESCRIPTION	MANDATORY
-inputtxt	Input hex file name.	Yes
-outputtext	Output Hash file name.	Yes
-igcks	Ignores line checksum while parsing the input hex file.	No
-filetype	Input hex file type.	Yes
-id	ID given to the hash for of each range.	No
-division	Specifies the division type.	No
-HSMfile	HSM file name.	No
-HSMconfig	HSM configuration type.	No
-tctype	Trust center type Eg. BOSCH TRUST CENTER, VW TRUST CENTER	No
-mode	Mode type eg. Online or Offline.	No
-oidpub	OID value.	No

c) INPUTSIGN

Generate signature for the given hex file and hash text file using MDG1TPROT.

Below is the syntax:

inputsign -inputhex [input hex file name] -inputtxt [input text file name] -outputtext [output sign text file name] -outputhex [output hex file name] -outputfiletype [output file type] -igcks [true or false] -memoryType [Memory type] -id [tag value] -filetype [input file type] -mode [bigendian or littleendian] -oidpub [OID value] -infoblockcount [infoblock count] -infoblockaddress [infoblock start address] -tctype [trust center type] -HSMfile [hsm file name] -HSMconfig [HSM configuration] -division[division type] -pinpath [pin file path] -dummysignature [Dummy signature value] -signaturealgo [Signature algorithm name]

SWITCHES	DESCRIPTION	MANDATORY
-inputhex	Input hex file name.	Yes
-outputhex	Output Hash file name.	Yes
-inputtxt	Input hex file name.	Yes
-outputtext	Output Hash file name.	No
-outputfiletype	Output hex file type.	Yes
-igcks	Ignores line checksum while parsing the input hex file.	Yes
-filetype	Input hex file type.	Yes
-id	ID given to the hash for of each range.	No
-memoryType	Memory type.	No
-division	Specifies the division type.	No
-pinpath	Pin file path.	No
-dummysignature	Dummy signature value.	No
-signaturealgo	Signature algorithm name.	No
-HSMfile	HSM file name.	No
-HSMconfig	HSM configuration type.	No
-tctype	Trust center type Eg. BOSCH TRUST CENTER, VW TRUST CENTER	No
-mode	Mode type eg. Online or Offline.	No
-oidpub	OID value.	No
-infoblockaddress	Infoblock start address.	No
-infoblockcount	Infoblock count for which hash value to be calculated.	No

Example:

1) inputsign -inputhex medc18.hex -filetype IHEX -outputhex outuput_sign1.hex -lyt 0x10 -outputfiletype IHEX -filemode LittleEndian -id tag1 -infoblockaddress 0x80020000 -inputtxt sign.txt

2) inputsign -inputhex medc18.hex -filetype IHEX -outputhex outuput_sign1.hex -lyt 0x10 -outputfiletype IHEX -filemode LittleEndian -id tag1 -infoblockaddress 0x80020000 -inputtxt sign.txt

inputsign -inputhex medc18.hex -filetype IHEX -outputhex outuput_sign2.hex -lyt 0x10 -outputfiletype IHEX -filemode LittleEndian -id tag2 -infoblockaddress 0x80020000 -inputtxt sign.txt

d) CERTINFO

Generate certificate keys information for the given hex file into output xml file.

Below is the syntax:

CERTINFO -inputfile [input hex file name] -mode [bigendian or littleendian] -hsm_file [HSM xml configuration file] -hsm_config [true or false to enable HSM configuration handling] -output [output xml file name] -infoblockaddr [infoblock start address]

SWITCHES	DESCRIPTION	MANDATORY
-inputfile	Input hex file name.	Yes
-mode	Mode type eg. bigendian or littleendian.	Yes
-hsm_file	HSM configuration xml file name.	No
-hsm_config	Boolean value true or false to enable HSM configuration handling.	No. Default value is false.
-infoblockaddress	Infoblock start address.	No when -hsm_config = true.
-output	Output xml file name.	Yes

Example:

1) CERTINFO -inputfile MG1CS080_02_40P_T0A00_HSM2.hex -mode LittleEndian -infoblockaddr 0x80000020 -output signInfo.xml
2) CERTINFO -inputfile HSM.hex -mode LittleEndian -hsm_file merged_corecfg_export.xml -hsm_config true -output signInfo.xml

e) SIGNHEX

Generates signature and certificate for the given hex file MDG1TPROT module

Below is the syntax :

signhex -inputhex [Input hex file name] -outputhex [Output Hash file name] -outputfiletype [Output hex file type.] -igcks [Ignores line checksum while parsing the input hex file.] -filetype [Input hex file type.] -id [ID given to the hash for of each range.] -memoryType [Memory type.] -division [Specifies the division type.] -pinpath [Pin file path.] -dummysignature [Dummy signature value.] -signaturealgo [Signature algorithm name.] -HSMfile [HSM file name.] -HSMconfig [HSM configuration type.] -tctype [Trust center type Eg. BOSCH TRUST CENTER, VW TRUST

CENTER] -mode [Mode type eg. Online or Offline.] -oidpub [OID public value.] -infoblockaddress [Infoblock start address.] -outputInfoBlkAddr [Output infoblock address after patching signature and certificate.] -lyt [layout of output hex file] -authbits [Authbits value -oid] -keypath[Keypath value] -filemode [Output file mode] -tswtype [TSW type] -lastrun [Last run value]

SWITCHES	DESCRIPTION	MANDATORY
-inputhex	Input hex file name.	Yes
-outputhex	Output Hash file name.	Yes
-outputfiletype	Output hex file type.	Yes
-igcks	Ignores line checksum while parsing the input hex file.	Yes
-filetype	Input hex file type.	Yes
-id	ID given to the hash for of each range.	No
-memoryType	Memory type.	No
-division	Specifies the division type.	No
-pinpath	Pin file path.	No
-dummysignature	Dummy signature value.	No
-signaturealgo	Signature algorithm name.	No
-HSMfile	HSM file name.	No
-HSMconfig	HSM configuration type.	No
-tctype	Trust center type Eg. BOSCH TRUST CENTER, VW TRUST CENTER	No
-mode	Mode type eg. Online or Offline.	No
-oidpub	OID value.	No
-infoblockaddress	Infoblock start address.	No
-infoblockaddress	Infoblock start address.	No
-outputInfoBlkAddr	Output infoblock address after patching signature and certificate.	No
-lyt	layout of output hex file	No
-authbits	Authbits value -oid	No
-keypath	Keypath value	No
-filemode	Output file mode	No
-tswtype	TSW type	No

-lastrun	Last run value	

Example:
1) signhex -inputhex mx18_make_INF.hex -filetype IHEX -igcks true -outputhex outuput_sign.hex -lyt 0x20 -outputfiletype IHEX -filemode LittleEndian -mode online -infoblockaddress 0x80000020

2) signhex -inputhex mx18_make_INF.hex -filetype IHEX -igcks true -outputhex outuput_sign.hex -lyt 0x20 -outputfiletype IHEX -filemode LittleEndian -mode online -infoblockaddress 0x80000020
signhex -inputhex mx18_make_INF.hex -filetype IHEX -igcks true -outputhex outuput_sign.hex -lyt 0x20 -outputfiletype IHEX -filemode LittleEndian -mode online -infoblockaddress 0x80000020

16) LOGFILE COMMAND:

Generates log file in the user defined path. When this command is invoked then default log file which is generated in the current folder will be deleted. This command should be present as the first command of all given commands.

Below is the syntax:

logfile -name [Log file path]

SWITCHES	DESCRIPTION	MANDATORY
-name	Log file path.	Yes

Example:
logfile -name .\log\hex.log read -in MG1CS001_P1112_V140.hex

17) SEGMENT_A_TRANSLATION COMMAND:

Performs translation of SEGMENT A addresses into SEGMENT 8 addresses. By default, HexmodX translates Segment A to Segment 8 address.
This command should be present before reading hex file using read command.

Below is the syntax:
segment_a_translation -segmentatosegment8 [true or false]

SWITCHES	DESCRIPTION	MANDATORY
-segmentatosegment8	Specify true to enable translation and false to disable translation.	Yes

Example:
1)read -in input01.hex
segment_a_translation -segmentAToSegment8 false
memCopy -from 0xA0010000 -l 0x20 -to 0x80020000 -o true
write -out _out/output.hex -lyt 0x10

18) VRTE FUNCTINALITIES COMMAND

a) Metadata Patching

Patches vrte infoblock metadata to hex file from the metadata XML file.

Below is the syntax :

vrte -metadata metadata_xml_file_name

SWITCHES	DESCRIPTION
-metadata	xml file which contains metadata for memory blocks in hex file.

Example:
1) vrte -metadata metadata.xml

b) MAC Calculation

Performs mac value generation for vrte memory blocks in hex file.

Below is the syntax :

vrte_mac -metadata metadata_xml_file
-keysfile keys_xml_file
[-mactype {SECURECALLBACK | BOOTMANAGERBLOCK}]
[-algo algorithm_name]

SWITCHES	DESCRIPTION
-metadata	xml file which contains metadata for memory blocks in | hex file.
-keysfile	specifies the key file.
-mactype	specify the mac type to perform sepcific mac generation
-algo	Algorithm name to perform mac generation

Example:
1) vrte_mac -metadata metadata.xml -keysfile keys.xml

2) vrte_mac -metadata metadata.xml -keysfile keys.xml -mactype SECURECALLBACK -algo AESCMAC

c) Signature Generation

Hash Generation

Performs sign text file generation from input hash text file.

Below is the syntax :

vrte_hash - metadata vrte_conf_file
- outputtxt output_hash_file
[-blockarea {HEADER | PAYLOAD}]
[-id tag_id]

[-ranges range_count]
[-from startaddress]
[-to endarress]
[-rangeoutput output_hash_for_ranges]

Example:

1. vrte_hash -id tag1 -outputtxt hash_cli0.txt -ranges 2 -FROM 0x20004000 -to 0x20005000 -FROM 0x20006000 -TO 0x20008000
2. vrte_hash -metadata metadata.xml -id tag1 -blockarea header -outputtxt hash_cli.txt

Sign Generation

```
Performs sign text file generation from input hash text file.

Below is the syntax :

       vrte_hashtosign     -inputtxt input_hash_file
                           -outputtxt output_sign_file
                           [-keypath keypath_for_sign_generation]
                           [-oid OID]
                           [-authbits authentication_bits]
                           [-mode {ONLINE | OFFLINE}]
                           [-dummy {TRUE | FALSE}]
                           [-type signature_type]
                           [-lastrun last_run]
                           [-pinpath sc_pin_path]
                           [-algo signature_algorithm]
----------------------------+-------------------------------------------------------------
   SWITCHES                 |    DESCRIPTION
----------------------------+-------------------------------------------------------------
   -metadata                |    xml file which contains metadata for memory blocks in
                            |     hex file.
   -keysfile                |    specifies the key file.
   -data_encryption         |    specify data encryption operation switch
   -algo                    |    Algorithm name to perform encryption
   -fill                    |    Fill pattern value to fill gaps if any present
----------------------------+-------------------------------------------------------------
Example:
   1. vrte_hashtosign -mode online -algo ECDSA -inputtxt hash.txt -outputtxt sign.txt
```

Signed Hex Generation

Performs sign text file generation from input hash text file.

Below is the syntax :

vrte_signedhex -inputtxt input_ sign_file
-metadata vrte_conf_file
[-infoblockaddr infoblock_address]
[-id tag_id]
[-blockarea {HEADER | PAYLOAD}]
[-mode {ONLINE | OFFLINE}]
[-dummy {TRUE | FALSE}]
[-type signature_type]
[-lastrun last_run]
[-pinpath sc_pin_path]

[-algo signature_algorithm]

Example:

1. vrte_signedhex -inputtxt sign.txt -metadata metadata.xml -blockarea payload -id tag1

d) Encryption

Performs encryption for vrte memory blocks in hex file.

Below is the syntax :

vrte_encryption -metadata metadata_xml_file
-keysfile keys_xml_file
[-data_encryption {TRUE | FALSE}]
[-algo algorithm_name]
[-fill {fill value}]

SWITCHES	DESCRIPTION
-metadata	xml file which contains metadata for memory blocks in hex file.
-keysfile	specifies the key file.
-data_encryption	specify data encryption operation switch
-algo	Algorithm name to perform encryption
-fill	Fill pattern value to fill gaps if any present

Example:

1) vrte_encryption -metadata metadata.xml -keysfile keys.xml

2) vrte_encryption -metadata metadata.xml -keysfile keys.xml -data_encryption TRUE -algo AES/CTR/NoPadding –fill 0xAA